Experiment – 01
Class Task

Name: Samridhi Singh
UID: 24BAI70514
Branch: BE-CSE(AIML)
Section: 24AIT_KRG-1_A
Semester: 5th
Subject: Advanced Database Management System

Question 1 (A):

A bank wants to classify all its accounts into three salary categories based on the monthly salary linked to each account:

"Low Salary": monthly salary strictly less than $20,000.

"Average Salary": monthly salary in the inclusive range [$20,000, $50,000].

"High Salary": monthly salary strictly greater than $50,000.

Write a PostgreSQL query to report the number of accounts in each category. All three categories must appear in the result, even if a category has zero accounts.

Solution:

```
SELECT
    c.category AS "Salary Category",
    COUNT(a.account_id) AS "Number of Accounts"
FROM (
    VALUES ('Low Salary'), ('Average Salary'), ('High Salary')
) AS c(category)
LEFT JOIN accounts a
    ON (c.category = 'Low Salary'     AND a.salary < 20000)
    OR (c.category = 'Average Salary' AND a.salary BETWEEN 20000 AND 50000)
    OR (c.category = 'High Salary'    AND a.salary > 50000)
GROUP BY c.category
ORDER BY CASE c.category
            WHEN 'Low Salary' THEN 1
            WHEN 'Average Salary' THEN 2
            WHEN 'High Salary' THEN 3
         END;
```

Explanation:

A VALUES list generates all three fixed categories first. A LEFT JOIN against the accounts table (instead of a plain GROUP BY on salary) guarantees every category is preserved, including ones with zero matching accounts, since COUNT(a.account_id) simply returns 0 for unmatched rows.

Question 2 (B):

Design an Employee Management System with Departments, Employees, and Salaries tables. Perform the following sequentially:

1. Create and populate the three tables.

2. Join the tables to view a department-wise breakdown.

3. Apply a 10% salary increase to all employees in the HR department, using an IN subquery.

4. Delete any salary record that drops strictly below $30,000.

5. List all remaining employees whose salary exceeds the company-wide average salary.

Solution:

```sql
-- 1. Create tables
CREATE TABLE departments (
    dept_id   SERIAL PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE employees (
    emp_id   SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id  INT REFERENCES departments(dept_id)
);

CREATE TABLE salaries (
    salary_id SERIAL PRIMARY KEY,
    emp_id    INT REFERENCES employees(emp_id),
    salary    NUMERIC
);

-- Populate tables
INSERT INTO departments (dept_name) VALUES ('HR'), ('IT'), ('Finance');

INSERT INTO employees (emp_name, dept_id) VALUES
    ('Alice', 1),
    ('Bob', 1),
    ('Charlie', 2),
    ('David', 3);

INSERT INTO salaries (emp_id, salary) VALUES
    (1, 25000),
    (2, 45000),
    (3, 60000),
    (4, 28000);

-- 2. Department-wise breakdown (multi-table join)
SELECT d.dept_name, e.emp_name, s.salary
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
JOIN salaries s  ON e.emp_id = s.emp_id
ORDER BY d.dept_name;

-- 3. 10% raise for HR employees (IN subquery filtering)
UPDATE salaries
SET salary = salary * 1.10
WHERE emp_id IN (
    SELECT e.emp_id
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    WHERE d.dept_name = 'HR'
);

-- 4. Delete salary records below $30,000
DELETE FROM salaries
WHERE salary < 30000;

-- 5. Employees earning above the company-wide average
```

```
SELECT e.emp_name, s.salary
FROM employees e
JOIN salaries s ON e.emp_id = s.emp_id
WHERE s.salary > (SELECT AVG(salary) FROM salaries);
```

Explanation:

The IN subquery first resolves the set of emp_id values belonging to the HR department by joining employees to departments, and the outer UPDATE applies the 10% raise only to salary rows whose emp_id falls in that set. The DELETE and final SELECT then operate on the updated salaries table.

Question 3 (C):

A pizza restaurant stores toppings and their costs in a table pizza_toppings. Every pizza must use exactly 3 different toppings, with no repeated toppings, and different orderings of the same 3 toppings must be treated as a single pizza. Write a PostgreSQL query to list every distinct 3-topping combination with its total cost, sorted by highest cost first, then alphabetically for ties.

Solution:

```
SELECT
    p1.topping_name || ', ' || p2.topping_name || ', ' || p3.topping_name AS
pizza,
    (p1.ingredient_cost + p2.ingredient_cost + p3.ingredient_cost) AS
total_cost
FROM pizza_toppings p1
JOIN pizza_toppings p2 ON p1.topping_name < p2.topping_name
JOIN pizza_toppings p3 ON p2.topping_name < p3.topping_name
ORDER BY total_cost DESC, pizza ASC;
```

Explanation:

Chaining the join conditions p1.topping_name < p2.topping_name < p3.topping_name forces each triple of toppings to be picked in strict alphabetical order. This automatically rules out repeated toppings within a pizza and collapses reordered duplicates (e.g. Chicken/Pepperoni/Sausage) down to a single combination.